

Strategy

lunes, 20 de abril de 2026 11:12

Es un patrón de **comportamiento**

Define una familia de algoritmos, **encapsular cada uno en un objeto y hacerlos intercambiables**

☞ Piensa en él como elegir la ruta en un GPS: puedes usar la estrategia "más rápida", "más corta" o "sin peajes", pero el viaje (el contexto) sigue siendo el mismo.

Usarlos cuando

Existen muchos algoritmos para **resolver una misma tarea**

Cada algoritmo necesita manejar su **propia información o lógica**

Se quiere evitar condicionales rígidos como if/else,witch

Estructura

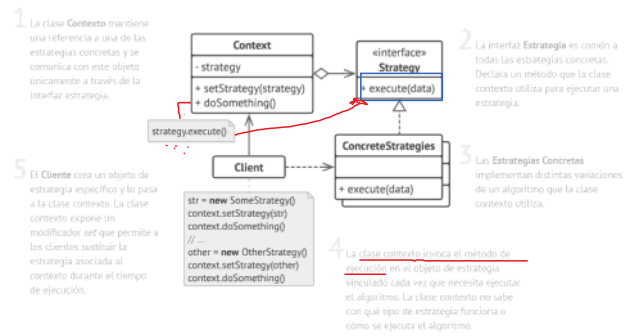
- **<Cliente>** El cliente generalmente crea las estrategias, pasa la strategy deseada a contexto
- **<Contexto>** delega el trabajo a la strategy seleccionada
- **<Strategy>(interface)** tiene el único método para disparar el algoritmo
- **<ConcreteStrategy>** tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas estrategias.

Ventajas

- Flexibilidad al poder cambiar algoritmos en tiempo de ejecución
- Agregar nuevas estrategias sin modifica el cliente

Desventajas

- El cliente debe conocer las estrategias
- Puede ver overhead en la comunicación de contexto y estrategia



Ejemplo

imagina que tienes un sistema de **ordenamiento**:

- Estrategia QuickSort
- Estrategia MergeSort
- Estrategia BubbleSort

El cliente solo sabe que necesita ordenar, pero no cómo. Según la situación (cantidad de datos, rendimiento esperado), se puede cambiar la estrategia en tiempo de ejecución.

State	Strategy
El comportamiento de un objeto depende del estado en el que se encuentre El patrón State es útil para una clase que debe realizar transiciones entre estados fácilmente.	Necesito uno de diferentes algoritmos opcionales para realizar una misma tarea El patrón Strategy es útil para permitir que una clase delegue la ejecución de un algoritmo a una instancia de una familia de estrategias
• En State, los diferentes estados: - son internos al contexto, - no los eligen las clases clientes - la transición se realiza entre los estados mismos	• En Strategy, las diferentes estrategias: - son conocidas desde afuera del contexto, por las clases clientes del contexto. - el Contexto del Strategy debe contener un mensaje público para cambiar el ConcreteStrategy.